BEGIN TODAY

Start here

Your first two hours

Run a program, predict its output, make a small change and complete the first two lesson quizzes. No game engine or electronics purchase is needed.

First-day plan / Tested starter code / Lessons 01.1–01.2

Prepared September 28, 2026

A guided self-study program with original lesson primers, assigned study, engineering labs, checkpoint projects and research work. Not an accredited degree or a substitute for the required practice and feedback.

A first session for today

Goal: run a small program, predict its behavior and explain the difference between input, state and output. No game engine, graphics SDK, electronics kit or paid account is needed today. The learner's age, prior programming and mathematics experience were not supplied, so this starting path assumes no programming experience and permits acceleration by demonstrated competence.

Extract the complete course ZIP into a folder you can find again. Open `CURRICULUM.html` in a browser for searchable lessons, or use the PDFs in `volumes`. The browser edition stores no information on a server. Assigned external readings still require internet access; they are linked, not copied into the package.

Time budget	Activity	Evidence to keep
0-15 minutes	Read Lesson 01.1 and trace its five movement commands on paper.	Your predicted positions.
15-40 minutes	Follow the setup instructions below and run `day01_motion.py`.	The command, Python version and output.
40-60 minutes	Explain every line using the walkthrough.	A short explanation in your own words.
60-80 minutes	Change speed to 10, predict the output, then run. Restore it and change the timestep to 0.5.	Predictions, observations and differences.
80-100 minutes	Complete the three questions after 01.1 and the three after 01.2 before reading the answer key.	Six answers, marked afterward.
100-120 minutes	Write a journal entry and optionally play the grid game.	One question to investigate next session.

These are session-planning estimates, not a deadline. Setup may take longer. Take breaks, stop at a working checkpoint and resume without treating slower progress as failure. Completing the reading and practical check matters more than matching the clock.

Open a shell and find the starter folder

A **terminal** is the window; a **shell** is the command interpreter inside it. PowerShell is a common Windows shell. Terminal is a common macOS terminal application. Linux has several terminal applications. Python is a different program launched from that shell.

Open the extracted `starter` folder in a terminal. Some file managers and editors offer an “Open in Terminal” option. Otherwise type `cd` followed by the quoted folder path and press Enter. The path must be the actual path on the learner's computer; do not copy a fictional username from an example.

On macOS/Linux, `pwd` shows the current folder and `ls` lists its files. In PowerShell, `Get-Location` and `Get-ChildItem` do the same. Confirm that `day01_motion.py` is listed before

continuing.

On **macOS or Linux**, first check the interpreter, then run the file:

```
python3 --version
python3 day01_motion.py
```

On **Windows**, a common Python launcher command is:

```
py -3 --version
py -3 day01_motion.py
```

When `py` is unavailable but `python --version` reports a supported Python 3 installation, use `python day01_motion.py`. Use one verified interpreter command consistently. Do not use Python 2. The supplied code was tested with Python 3.13.5; that is the tested environment, not a claim that this particular release is always the newest or the only usable release.

When Python is missing, use the official Python downloads and setup documentation, then reopen the terminal and repeat the version check. Official entry points: Python downloads and Python setup and usage. On a managed computer, use the approved installation process rather than changing security settings. You can complete the paper trace while installation is arranged. [R04]

The first program, explained

Open `starter/day01_motion.py` in a plain-text code editor. Save changes to that file, not to a word-processing document or a file ending in `.py.txt`.

```
position_units = 0.0
speed_units_per_second = 6.0
timestep_seconds = 0.25

for tick in range(1, 6):
    position_units = position_units + speed_units_per_second * timestep_seconds
    print(f"tick {tick}: x={position_units:.2f} units")
```

`position_units = 0.0` creates the starting position. An assignment evaluates the right side and binds the result to the name on the left. It does not make an algebraic equation that must remain true forever.

`speed_units_per_second = 6.0` says the player travels six units during one second. `timestep_seconds = 0.25` makes each update represent one quarter of a second. The names carry units so the calculation can be checked.

`range(1, 6)` supplies 1, 2, 3, 4 and 5. The upper endpoint is excluded. The indented lines execute once for each value. The movement line adds `6.0 * 0.25`, or 1.5 units, to the previous position. The print line displays the current state; it does not move the player. The formatting `:.2f` displays two digits after the decimal point. It changes the text presentation, not the underlying movement rule. [R03, R04]

Expected output:

```
tick 1: x=1.50 units
tick 2: x=3.00 units
```

```
tick 3: x=4.50 units
tick 4: x=6.00 units
tick 5: x=7.50 units
```

Before each change, write the five expected positions. With speed 10 and the original timestep, each update adds 2.5 units and the last position is 12.5. With the original speed 6 and timestep 0.5, each update adds 3 units and the last position is 15. Change one variable at a time so a disagreement has a clear cause.

Common first-day problems

Symptom	What to check
Command not found, or Python is not recognized	Confirm Python is installed through the approved route; reopen the shell; check the available launcher command.
Cannot open file	List the current directory. Confirm the exact filename and that the terminal is inside `starter`.
A >>> prompt is visible	That is the interactive Python prompt, not the ordinary shell. Enter `exit()` to return, then run the file command in the shell.
Indentation error	Preserve consistent indentation beneath the `for` line. Do not mix arbitrary tabs and spaces.
Syntax error	Read the indicated line and the preceding one. Check quotes, parentheses and the colon after `for`.
Program runs but changes do not appear	Save the edited file and confirm that the command runs that same file, not another copy.

Never respond to a setup error by running an unexplained administrator command, deleting system folders, disabling protections or sharing passwords. The supplied starter programs need none of those actions.

A playable reward and later test commands

After the motion exercise, run the tiny terminal game:

```
python3 grid_game.py
```

Use `py -3` instead of `python3` on Windows when that is the verified launcher. Move with W/A/S/D to reach G; Q exits. A wall or boundary blocks movement but still consumes a turn. The supplied game uses classes and dataclasses that are taught later; playing it today does not require understanding every implementation detail.

The following command replays a complete route:

```
python3 grid_game.py --replay SSSSDDDDDD
```

The demonstration reaches position (6,4) on turn 10. Find a different route after learning the rules. In Module 04, run the supplied automated tests and write additional tests of your own:

```
python3 -m unittest discover -v
```

The package's 23 Python tests and C++ motion-contract test passed in the recorded Linux environment. They validate the supplied starter, not the learner's future work and not every operating system. See `starter/TESTED_ENVIRONMENT.md` for the exact scope.

A two-week starting rhythm

This is a sample at roughly 10-15 study hours per week, not a promise that every reading or lab will fit. Repeat a session when the practical check is incomplete. Do not skip a concept merely to keep the dates aligned.

Session	Focus
Day 1	Lessons 01.1-01.2, paper trace, motion program and six quiz questions.
Day 2	Lesson 01.3: predict assignments before execution; repair a failed swap.
Day 3	Lesson 01.4: units, frame intervals and an engineering journal.
Day 4	Lesson 02.1: values, types and text-to-number conversion.
Day 5	Review Module 01; explain the motion program without looking at notes.
Days 6-7	Optional gentle practice, catch-up or rest. No new topic quota.
Day 8	Lesson 02.2: conditions and boundary tests.
Day 9	Lesson 02.3: loops, progress and invariants.
Day 10	Lesson 02.4: functions, return values and input contracts.
Day 11	Repair missed questions and add your own function test cases.
Day 12	Start Module 02's small resource-gathering simulation.
Day 13	Review the module project; demonstrate one unfamiliar change.
Day 14	Catch-up, planning and rest before Module 03.

Begin each later session with a five-minute recall exercise. End with the exact command that runs the current project, one unresolved question and the next smallest testable step. Use `templates/LAB_JOURNAL.md` rather than relying on memory.

How to know a lesson is learned

Answer the three questions **before** opening the answer key. Award 0 for incorrect/missing, 1 for partly correct or unsupported and 2 for correct with a clear explanation. A suggested formative threshold is at least **5/6**, plus the lesson's practical pass condition and completed assigned study. A quiz score alone does not establish engineering competence.

For a missed question, identify the mistaken idea, solve a changed example and retest after a delay. Do not merely copy the answer. A reviewer should occasionally ask for an unfamiliar variation or a live code change. AI can explain or critique, but the learner must predict, test and defend the result independently.

Next: complete the two full lessons included on the following pages, then continue with Volume 1. `PROGRAM_GUIDE.pdf` contains placement, prerequisites, hour estimates, all ten checkpoint projects, graduate expectations and the resource directory. `ANSWER_KEY.pdf` is deliberately separate from the printable lessons.

LESSON 01.1 • MODULE 01

What a program actually does

Learning goal Explain state, instructions, input and output using a tiny game.

A program is a set of instructions that transforms information. Its state is the information that must persist: a player position, a health value, or a list of enemies. An input is a new observation or command. An output might be a picture, sound, file or network message. A game repeatedly reads inputs, changes state and produces outputs. The picture is a representation of the state, not the state itself.

A processor executes machine instructions; an operating system manages resources and access to devices. A game engine is software that organizes recurring tasks such as rendering and asset loading. A graphics processor is hardware that executes suitable workloads. Keep these layers distinct: changing a game rule is not the same activity as designing a processor.

WORKED EXAMPLE

For a player at x = 2, the command move-right by 3 units produces x = 5. Drawing the character twice must not move it twice. Rendering observes state; the simulation changes it.

PRACTICE + PASS CRITERION

On paper, trace five commands: right 3, left 1, right 0, left 4, right 2, starting at x = 2. Draw boxes for input, simulation state and output. Save the trace. Pass: every state transition is explainable without running code.

ASSIGNED STUDY

R03: introductory functions/variables lecture; R15: Basics of Information.

CHECK YOUR UNDERSTANDING

1. What is the difference between state and a displayed image?

2. What final position results from the five lab commands?

3. Why should drawing a character not update its position?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 01.2 • MODULE 01

Files, terminals and reproducible execution

Learning goal Run a saved Python program and distinguish a shell from an interpreter.

A source file stores the instructions you write. A terminal presents a command-line interface; its shell starts programs and manages paths. The Python interpreter is a program that runs Python instructions. A shell prompt is not a Python prompt. Commands such as cd belong in the shell; expressions such as print(2 + 3) belong in Python code.

A working directory tells a process where relative paths begin. Many apparent programming errors are actually wrong-directory or wrong-file errors. Reproducibility starts by recording the exact command, file and tool version. Use a dedicated course folder. Do not run copied administrative commands or upload private files merely to make a tutorial work.

WORKED EXAMPLE

With day01_motion.py in the current folder, python3 day01_motion.py runs that file on many macOS/Linux setups. On Windows, py -3 day01_motion.py is a common launcher form. First confirm the available command with its --version option.

PRACTICE + PASS CRITERION

Follow START_HERE. Run the included program twice from the same folder; save the command and output in your journal. Intentionally misspell the filename, read the error, then repair it. Pass: you can explain which program produced the error.

ASSIGNED STUDY

R04: Using the Python Interpreter; installation links in START_HERE.

CHECK YOUR UNDERSTANDING

1. What does a relative filename depend on?

2. Is cd a Python statement in an ordinary script?

3. What three facts help another person reproduce a run?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.